

Unidad VI: Árboles Avanzados

Clase 11 – Árbol B: estructuras para grandes volúmenes

Objetivo

Comprender estructuras usadas en sistemas reales (bases de datos).

Tema

- Motivación (accesos a disco).
- Estructura de Árbol B.
- Diferencias con ABB y AVL.

Árbol B: estructuras de búsqueda para grandes volúmenes de datos

Introducción conceptual

Las estructuras de árbol permiten organizar información de manera jerárquica para facilitar operaciones como búsqueda, inserción, eliminación y recorrido ordenado. En árboles binarios de búsqueda, cada nodo contiene una clave y divide el espacio de búsqueda en dos caminos posibles: valores menores y valores mayores. Esta idea resulta muy eficiente cuando los datos se encuentran en memoria principal y el costo de acceder a cada nodo es bajo.

Sin embargo, muchos sistemas reales trabajan con volúmenes de información que no pueden mantenerse completamente en memoria. Bases de datos, sistemas de archivos, plataformas bancarias, motores de búsqueda, sistemas logísticos y aplicaciones empresariales suelen almacenar información en disco, unidades de estado sólido o estructuras paginadas. En esos contextos, el problema no es solamente ordenar los datos o mantener el árbol balanceado. El problema central es reducir la cantidad de accesos físicos necesarios para encontrar la información.

El Árbol B surge como una respuesta a ese escenario. Su diseño permite agrupar varias claves dentro de un mismo nodo y utilizar múltiples caminos de búsqueda. De esta manera, se reduce la altura del árbol y se aprovecha mejor cada bloque de almacenamiento leído. Por eso, el Árbol B no debe entenderse como una simple variante del árbol binario, sino como una estructura pensada para manejar grandes volúmenes de datos con eficiencia en sistemas reales.

Del árbol binario al problema del almacenamiento

Un árbol binario de búsqueda, también conocido como ABB, organiza los datos bajo una regla simple: para cada nodo, los valores menores se ubican en el subárbol izquierdo y los valores mayores en el subárbol derecho. Esta estructura permite

búsquedas ordenadas y recorridos eficientes cuando el árbol mantiene una altura razonable.

El problema aparece cuando el árbol se desbalancea. Si los datos se insertan en un orden desfavorable, el ABB puede degenerar en una estructura similar a una lista. En ese caso, buscar un valor puede requerir recorrer muchos nodos, elevando el costo temporal desde un comportamiento esperado cercano a $O(\log n)$ hacia $O(n)$, donde n representa la cantidad de elementos almacenados.

El árbol AVL mejora esta situación. Un AVL es un árbol binario de búsqueda balanceado, es decir, una estructura que controla la diferencia de altura entre los subárboles izquierdo y derecho de cada nodo. Cuando esa diferencia supera ciertos límites, el árbol se reorganiza mediante rotaciones. Gracias a ese mecanismo, el AVL conserva una altura logarítmica y mantiene eficientes sus operaciones principales.

No obstante, el AVL sigue siendo binario. Cada nodo contiene una clave y como máximo dos hijos. Esta característica puede ser adecuada para memoria principal, pero presenta una limitación cuando los datos se almacenan en disco. Si cada nodo implicara leer una página o bloque de almacenamiento, recorrer muchos niveles sería costoso. Aunque el AVL esté balanceado, su estructura sigue obligando a bajar por una secuencia de nodos pequeños.

En sistemas de almacenamiento, leer un bloque completo suele ser mucho más eficiente que leer muchos puntos aislados. Por ese motivo, conviene que cada nodo contenga varias claves y permita decidir entre varios caminos. Esa es la idea fundamental del Árbol B.

El costo real de acceder a disco

En análisis de algoritmos suele hablarse de costo temporal usando notación O , como $O(1)$, $O(\log n)$ u $O(n)$. Esta notación permite comparar cómo crece el tiempo de ejecución a medida que aumenta la cantidad de datos. Sin embargo, en estructuras pensadas para bases de datos o archivos, también importa el costo de entrada/salida, conocido habitualmente como I/O, del inglés Input/Output. Este costo representa las operaciones de lectura y escritura sobre almacenamiento externo.

Acceder a memoria principal es muy rápido en comparación con acceder a disco. Aunque las unidades de estado sólido modernas han reducido mucho esa diferencia, el principio sigue siendo válido: leer menos páginas o bloques suele mejorar el rendimiento.

Una base de datos, por ejemplo, no trabaja necesariamente leyendo registro por registro. Muchas veces organiza la información en páginas. Una página puede contener varias claves, referencias o registros. Si una estructura obliga a leer una página para obtener una sola clave, se desaprovecha gran parte del bloque leído. En cambio, si cada nodo del árbol contiene muchas claves, una sola lectura permite realizar varias comparaciones y decidir entre múltiples caminos.

Por eso, el Árbol B se diseña pensando en páginas o bloques. Cada nodo puede interpretarse como una unidad de almacenamiento relativamente grande, capaz de contener varias claves ordenadas y varias referencias a hijos.

Árbol B como árbol multicamino

Un Árbol B es una estructura de búsqueda balanceada y multicamino. La palabra multicamino indica que un nodo puede tener más de dos hijos. A diferencia de un ABB o un AVL, donde cada nodo divide la búsqueda en dos ramas, un nodo de Árbol B puede dividirla en varios rangos.

Un nodo con varias claves ordenadas puede representarse conceptualmente así:

[20 | 40 | 60]

Este nodo no conduce solamente hacia izquierda o derecha. Permite separar cuatro rangos posibles:

Valores menores que 20.

Valores entre 20 y 40.

Valores entre 40 y 60.

Valores mayores que 60.

La estructura sigue conservando una lógica ordenada. Las claves dentro del nodo están en orden creciente y los hijos representan intervalos de valores. La diferencia es que la decisión de búsqueda no se limita a dos opciones, sino que puede elegir entre varios caminos.

En un sistema de e-commerce, por ejemplo, podría existir un índice por código de producto. Un nodo de Árbol B podría contener claves separadoras como:

[1000 | 2500 | 5000 | 8000]

Si se busca el producto 4300, el sistema no necesita recorrer secuencialmente todos los productos. Dentro del nodo identifica que 4300 pertenece al rango entre 2500 y 5000, y continúa por el hijo correspondiente. Con una sola lectura de nodo, se descartaron varios rangos completos.

Propiedades fundamentales del Árbol B

El Árbol B mantiene ciertas propiedades que garantizan su eficiencia y su orden interno. Cada nodo puede contener varias claves, pero esas claves deben mantenerse ordenadas. Además, cada nodo puede tener varios hijos, y esos hijos deben representar correctamente los rangos definidos por las claves del nodo.

Una propiedad importante es que todas las hojas quedan al mismo nivel. Esto significa que el árbol se mantiene balanceado. No hay ramas extremadamente largas y otras muy cortas, como podría ocurrir en un ABB desbalanceado.

También existe una cantidad máxima de claves por nodo. Cuando un nodo se llena y se intenta insertar una clave adicional, el nodo debe dividirse. Esa división produce una reorganización: una clave central asciende al padre y las claves restantes se distribuyen en dos nodos. Este proceso permite que el árbol crezca de manera controlada y conserve baja altura.

En términos prácticos, el Árbol B busca tres objetivos al mismo tiempo: mantener las claves ordenadas, sostener el balance del árbol y reducir la cantidad de niveles necesarios para llegar a los datos.

Orden del Árbol B

El orden de un Árbol B define, de manera simplificada, cuántos hijos puede tener como máximo cada nodo. Si un Árbol B es de orden m , entonces cada nodo puede tener hasta m hijos y hasta $m - 1$ claves.

Por ejemplo, en un Árbol B de orden 5:

Cada nodo puede tener como máximo 5 hijos.

Cada nodo puede tener como máximo 4 claves.

Si se intenta insertar una quinta clave en un nodo, el nodo queda excedido y debe dividirse.

Esta relación entre hijos y claves es importante porque explica la naturaleza multicamino del Árbol B. Un nodo con 4 claves puede separar hasta 5 rangos de búsqueda. En comparación, un nodo de ABB o AVL contiene una sola clave y permite separar solamente dos rangos.

En implementaciones reales, el orden puede ser mucho mayor. Una base de datos puede configurar nodos o páginas que almacenen decenas, cientos o incluso más claves, dependiendo del tamaño de página, del tipo de clave y de la estructura del índice. Cuanto más claves útiles entren en un nodo, menor será la altura del árbol y menor la cantidad de accesos necesarios.

Búsqueda en un Árbol B

La búsqueda en un Árbol B combina dos operaciones: buscar dentro del nodo actual y luego elegir el hijo adecuado si la clave no se encuentra allí.

Supongamos un nodo con las claves:

[20 | 40 | 60]

Si se busca el valor 40, la búsqueda termina en ese nodo, porque la clave fue encontrada.

Si se busca el valor 35, el sistema compara con las claves del nodo y determina que 35 se encuentra entre 20 y 40. Por lo tanto, continúa por el hijo ubicado entre esas dos claves.

Si se busca el valor 75, el sistema determina que es mayor que 60 y continúa por el hijo derecho correspondiente al rango de valores mayores.

El proceso se repite hasta encontrar la clave o llegar a una hoja donde la clave no está. Si se llega a una hoja y la clave no aparece, la búsqueda finaliza indicando que el valor no pertenece al árbol.

La eficiencia del Árbol B no depende solamente de comparar rápido dentro de un nodo. Su principal ventaja está en que cada nodo permite descartar muchos rangos de una vez. En un contexto de almacenamiento paginado, esto significa menos lecturas y menor costo de acceso.

Comparación entre ABB, AVL y Árbol B

El ABB, el AVL y el Árbol B son estructuras de búsqueda, pero no responden exactamente al mismo problema.

El ABB organiza datos de forma ordenada mediante una regla binaria. Es simple y útil, pero puede desbalancearse si no se controla su forma.

El AVL conserva la lógica del ABB, pero agrega balance automático. Su objetivo es evitar que el árbol crezca de manera irregular. Para eso utiliza rotaciones y mantiene la altura bajo control.

El Árbol B también mantiene una estructura ordenada y balanceada, pero su diseño está orientado a reducir niveles mediante nodos con múltiples claves. No se limita a dos hijos, sino que permite varios caminos desde un mismo nodo.

La diferencia puede resumirse de esta forma:

El ABB ordena, pero puede desbalancearse.

El AVL ordena y balancea, pero sigue siendo binario.

El Árbol B ordena, balancea y reduce accesos agrupando muchas claves por nodo.

Esta distinción es clave. No se elige un Árbol B porque “sea más moderno” o porque “reemplace” al AVL en todos los casos. Se lo elige cuando el volumen de datos y el modelo de almacenamiento hacen conveniente reducir accesos a nodos o páginas.

Inserción en un Árbol B

Insertar una clave en un Árbol B requiere conservar el orden interno de los nodos y las propiedades generales del árbol. La operación comienza buscando la hoja donde la clave debería ubicarse. Si esa hoja tiene espacio disponible, la clave se inserta en orden dentro del nodo.

Por ejemplo, si una hoja contiene:

[10 | 20 | 40]

e insertamos 30, el nodo queda:

[10 | 20 | 30 | 40]

No se rompe ninguna regla, porque las claves siguen ordenadas y el nodo no supera el máximo permitido.

El caso interesante aparece cuando el nodo ya está lleno. Supongamos un Árbol B de orden 5, donde cada nodo puede contener como máximo 4 claves. Si una hoja contiene:

[5 | 10 | 20 | 30]

e insertamos 15, el resultado temporal sería:

[5 | 10 | 15 | 20 | 30]

Ese nodo ya no es válido porque contiene 5 claves y el máximo permitido es 4. Para corregir la situación, el nodo se divide. La clave central se promociona al padre y las claves menores y mayores quedan separadas en dos nodos.

En este ejemplo, la clave central es 15. Después de la división:

15 sube al padre.

[5 | 10] queda como nodo izquierdo.

[20 | 30] queda como nodo derecho.

Si el nodo dividido era la raíz, se crea una nueva raíz con la clave promocionada:

[15]

[5 | 10] [20 | 30]

Este mecanismo se conoce como división y promoción. Es el corazón de la inserción en Árbol B.

División y promoción de claves

La división se produce cuando un nodo excede su capacidad máxima. Para resolverlo, se toma una clave central, se la promociona al padre y se separan las claves restantes en dos nodos.

La promoción no significa que la clave “desaparece” del orden general. Al contrario, pasa a cumplir una función de separación entre dos rangos. Las claves menores quedan en el hijo izquierdo y las mayores en el hijo derecho.

Supongamos que un nodo queda temporalmente así:

[6 | 10 | 15 | 20 | 30]

Si se toma como clave media 15, la división genera:

Clave promocionada: 15.

Nodo izquierdo: [6 | 10].

Nodo derecho: [20 | 30].

El padre utilizará 15 como clave separadora entre ambos hijos.

Este proceso puede propagarse. Si el padre también se llena al recibir la nueva clave, el padre también debe dividirse. Si la división llega hasta la raíz y la raíz se llena, se crea una nueva raíz. De esta manera, el Árbol B crece hacia arriba, no hacia abajo de manera descontrolada.

Ejemplo aplicado de inserción

Consideremos un Árbol B de orden 5. Cada nodo puede tener como máximo 4 claves.

Se insertan los valores:

10, 20, 5, 6, 12

Primero se inserta 10:

[10]

Luego 20:

[10 | 20]

Luego 5:

[5 | 10 | 20]

Luego 6:

[5 | 6 | 10 | 20]

Hasta este punto el nodo está lleno, pero sigue siendo válido porque contiene 4 claves.

Al insertar 12, el nodo quedaría temporalmente:

[5 | 6 | 10 | 12 | 20]

Como se supera el máximo permitido, el nodo se divide. La clave central es 10. Esa clave se promociona y se crea una nueva raíz:

[10]

[5 | 6] [12 | 20]

Este ejemplo muestra cómo el Árbol B controla su crecimiento. En lugar de generar una rama larga, divide el nodo lleno y aumenta la capacidad de decisión del nivel superior.

Si luego se inserta 30, el valor va al hijo derecho porque es mayor que 10:

[10]

[5 | 6] [12 | 20 | 30]

Si se inserta 7, va al hijo izquierdo porque es menor que 10:

[10]

[5 | 6 | 7] [12 | 20 | 30]

El árbol conserva baja altura y mantiene todas las hojas al mismo nivel.

Por qué el Árbol B reduce la altura

La altura de un árbol representa la cantidad de niveles que deben recorrerse desde la raíz hasta las hojas. En estructuras de búsqueda, una menor altura suele implicar menos pasos para encontrar un dato.

En un árbol binario, cada nodo permite elegir entre dos caminos. Por lo tanto, aunque el árbol esté balanceado, la cantidad de niveles crece en función de divisiones binarias.

En un Árbol B, cada nodo puede contener muchas claves y muchos hijos. Eso significa que cada nivel puede dividir el conjunto de datos en muchas partes. Cuanto mayor sea la cantidad de claves por nodo, menor será la altura necesaria para almacenar una gran cantidad de elementos.

Esta propiedad es especialmente valiosa en bases de datos. Si una búsqueda en un AVL requiere recorrer 20 niveles, podría implicar hasta 20 accesos a páginas distintas. Si un Árbol B equivalente resuelve la búsqueda en 3 o 4 niveles, la diferencia práctica es enorme.

La complejidad teórica de búsqueda en un Árbol B también es logarítmica, pero el punto clave no es solo $O(\log n)$. La ventaja principal es que el logaritmo tiene una base mucho mayor, porque cada nodo puede abrir muchos caminos. En términos prácticos, esto reduce la cantidad de accesos a almacenamiento.

Eliminación en Árbol B

La eliminación en un Árbol B es más compleja que la inserción porque no alcanza con quitar una clave. Después de eliminar, el árbol debe seguir cumpliendo sus propiedades: claves ordenadas, hijos asociados a rangos correctos, hojas al mismo nivel, balance general y cantidad mínima de claves por nodo.

Cuando se elimina una clave, puede aparecer un problema llamado subocupación, también conocido como underflow. La subocupación ocurre cuando un nodo queda con menos claves que las mínimas permitidas. En ese caso, el árbol debe reorganizarse para recuperar su estructura válida.

Si la clave a eliminar está en una hoja, el caso puede ser simple. Por ejemplo, si una hoja contiene:

[10 | 20 | 30]

y se elimina 20, queda:

[10 | 30]

Si el nodo conserva una cantidad válida de claves, la operación termina. Pero si el nodo queda con muy pocas claves, hay que corregir la estructura.

La primera alternativa es el préstamo o redistribución. Consiste en tomar una clave de un hermano que tenga claves suficientes. El padre participa como intermediario porque sus claves separan los rangos entre los hijos. No se trata simplemente de mover un valor de un nodo a otro; el movimiento debe preservar el orden global del árbol.

Por ejemplo, supongamos un padre con la clave:

[20]

y dos hijos:

[5] [30 | 40]

Si el hijo izquierdo necesita una clave y el hijo derecho puede prestar, el padre puede intervenir para redistribuir los valores. Una clave del padre baja hacia el hijo con pocas claves, y una clave del hermano sube al padre. El objetivo es mantener la separación correcta de rangos.

Cuando ningún hermano tiene claves suficientes para prestar, se realiza una fusión. La fusión une el nodo con pocas claves, una clave separadora del padre y un hermano. Por ejemplo:

Padre:

[20]

Hijos:

[5] [30]

Una fusión puede producir:

[5 | 20 | 30]

En este caso, el padre pierde una clave. Si el padre queda con pocas claves, el problema puede propagarse hacia arriba. Por eso la eliminación puede afectar más de un nivel del árbol.

Si la clave a eliminar está en un nodo interno, no se la quita dejando un hueco, porque esa clave separa subárboles. Se la reemplaza por una clave equivalente que conserve el orden, normalmente el predecesor o el sucesor. El predecesor es la mayor clave del subárbol izquierdo. El sucesor es la menor clave del subárbol derecho. Luego se elimina esa clave reemplazante desde la hoja donde estaba originalmente.

Este comportamiento se parece al caso de eliminación en un ABB, pero con una dificultad adicional: después de eliminar en la hoja, puede aparecer subocupación y será necesario aplicar préstamo o fusión.

La eliminación no cambia la idea central del Árbol B, pero evidencia que mantener balance, orden y ocupación mínima requiere una administración cuidadosa de los nodos.

Árbol B+ como variante aplicada a bases de datos

El Árbol B+ es una variante del Árbol B muy utilizada en bases de datos e índices. Conserva la idea de nodos con múltiples claves y baja altura, pero organiza la información de manera diferente.

En un Árbol B, las claves pueden estar en nodos internos y hojas, y pueden estar asociadas a datos en distintos niveles. En un Árbol B+, los datos reales se concentran en las hojas. Los nodos internos funcionan principalmente como índices o guías de navegación.

Esta diferencia es fundamental. En un Árbol B+, los nodos internos ayudan a llegar rápidamente a la hoja correcta, pero los registros o referencias principales se

encuentran en el nivel de hojas. Además, las hojas suelen estar enlazadas entre sí, lo que facilita recorridos ordenados.

Esta característica vuelve al Árbol B+ especialmente útil para búsquedas por rango.

Por ejemplo, en una base de datos bancaria, podría ser necesario obtener todas las operaciones entre dos fechas. En un Árbol B+ indexado por fecha, el sistema puede buscar la primera hoja donde aparece la fecha inicial y luego avanzar secuencialmente por las hojas enlazadas hasta llegar a la fecha final.

Lo mismo ocurre en un sistema de auditoría que necesita revisar facturas entre dos importes, en una plataforma de educación que consulta inscripciones por rango de legajos, o en una empresa logística que busca envíos programados entre dos fechas.

En estos casos, no alcanza con encontrar un dato puntual. Se necesita recorrer un intervalo ordenado. El Árbol B+ facilita ese recorrido porque sus hojas pueden funcionar como una secuencia ordenada.

Diferencias entre Árbol B y Árbol B+

El Árbol B y el Árbol B+ comparten la idea de reducir altura mediante nodos con múltiples claves, pero se diferencian en la organización de los datos.

En el Árbol B, una clave ubicada en un nodo interno puede formar parte directa de la estructura de datos buscada. En el Árbol B+, los nodos internos se utilizan como estructura de navegación, mientras que los datos efectivos se ubican en las hojas.

En el Árbol B, una búsqueda puntual puede terminar en un nodo interno si la clave está allí. En el Árbol B+, la búsqueda normalmente continúa hasta una hoja, porque las hojas contienen las referencias principales a los datos.

En el Árbol B+, las hojas enlazadas permiten recorridos por rango de manera muy eficiente. Esta característica lo hace especialmente adecuado para índices de bases de datos, donde son frecuentes las consultas del tipo “entre dos fechas”, “entre dos códigos”, “mayores que cierto importe” o “ordenados por determinada clave”.

Una forma práctica de diferenciarlos es la siguiente:

El Árbol B reduce altura agrupando claves en nodos multicamino.

El Árbol B+ usa los nodos internos como índice y concentra los datos en hojas enlazadas.

Por eso, el Árbol B+ suele ser más conveniente cuando la aplicación necesita muchas consultas ordenadas o por rango.

Aplicaciones reales del Árbol B y B+

Los Árboles B y B+ aparecen en sistemas donde el volumen de datos y el costo de acceso hacen necesario optimizar la búsqueda.

En bases de datos, se utilizan como fundamento conceptual de muchos índices. Cuando se crea un índice sobre un campo como `id_cliente`, `fecha_operacion`, `codigo_producto` o `numero_factura`, el motor necesita una estructura que permita localizar valores sin recorrer toda la tabla. Una estructura tipo B o B+ permite resolver esa búsqueda de manera eficiente.

En sistemas bancarios, pueden utilizarse índices por número de cuenta, fecha de transacción o identificador de operación. En plataformas de e-commerce, pueden utilizarse para buscar productos por código, órdenes por fecha o clientes por identificador. En logística, pueden ayudar a consultar envíos por número de seguimiento, fecha estimada o zona de distribución.

La utilidad no se limita a encontrar un único valor. En muchos sistemas empresariales son frecuentes las consultas por rango. Por ejemplo:

Operaciones realizadas entre dos fechas.

Facturas con importe entre dos valores.

Productos con código dentro de cierto intervalo.

Clientes registrados durante un período determinado.

En estos escenarios, el Árbol B+ resulta especialmente relevante porque sus hojas enlazadas facilitan recorrer datos ordenados sin volver a subir y bajar constantemente por el árbol.

Criterios para elegir una estructura de árbol

La elección entre ABB, AVL, Árbol B o Árbol B+ no debe hacerse por memorización, sino por el problema que se quiere resolver.

Un ABB puede ser suficiente cuando se necesita una estructura simple de búsqueda ordenada y el volumen de datos es moderado, siempre que no exista riesgo importante de desbalance o que el orden de inserción no perjudique la estructura.

Un AVL es adecuado cuando se trabaja en memoria y se necesita garantizar que el árbol se mantenga balanceado. Es útil cuando se requieren búsquedas eficientes y se acepta el costo adicional de las rotaciones durante inserciones o eliminaciones.

Un Árbol B es conveniente cuando los datos son numerosos y el costo de acceder a nodos individuales es relevante. Su diseño reduce la altura y aprovecha mejor cada bloque leído.

Un Árbol B+ es especialmente útil cuando, además de búsquedas puntuales, se requieren recorridos ordenados y consultas por rango. Por eso se asocia fuertemente con índices de bases de datos.

Un error frecuente consiste en creer que todas estas estructuras compiten en igualdad de condiciones. En realidad, cada una responde a restricciones distintas. El AVL resuelve muy bien el problema del balance en árboles binarios en memoria. El Árbol B resuelve mejor el problema del acceso eficiente a grandes volúmenes almacenados en bloques.

Errores frecuentes de interpretación

Uno de los errores más comunes es pensar que el Árbol B es simplemente un ABB con nodos más grandes. Esta interpretación es incompleta. El Árbol B no solo agrupa claves; también redefine la forma de dividir el espacio de búsqueda mediante múltiples hijos y mantiene reglas de ocupación, división y balance.

Otro error frecuente es creer que si una estructura tiene costo $O(\log n)$, entonces todas las estructuras logarítmicas son equivalentes. En sistemas reales, dos estructuras con comportamiento logarítmico pueden tener rendimientos muy distintos. Un AVL y un Árbol B pueden ser eficientes desde el punto de vista teórico, pero el Árbol B puede requerir muchos menos accesos físicos cuando los datos están en disco.

También puede confundirse el Árbol B con el Árbol B+. La diferencia central es que en el B+ los datos se concentran en hojas y los nodos internos se usan como índice. Esa organización favorece las búsquedas por rango y los recorridos ordenados.

Otro punto importante es no reducir el Árbol B a la operación de inserción. La inserción permite entender la división y promoción de claves, pero la estructura completa también contempla búsqueda, eliminación, balance, ocupación mínima y reorganización de nodos.

Representación conceptual en Java

Desde el punto de vista de implementación, un Árbol B puede representarse mediante nodos que contienen un arreglo de claves y un arreglo de hijos. A diferencia de un nodo binario tradicional, donde suele existir una referencia al hijo izquierdo y otra al hijo derecho, un nodo de Árbol B necesita almacenar múltiples referencias.

Una representación conceptual de un nodo podría incluir:

Un arreglo de claves enteras o genéricas.

Un arreglo de referencias a hijos.

Una variable que indique cuántas claves reales contiene el nodo.

Una marca que indique si el nodo es hoja.

Por ejemplo, en una implementación didáctica en Java, un nodo podría tener esta forma conceptual:

```
class NodoB {
    int[] claves;
    NodoB[] hijos;
    int cantidadClaves;
    boolean hoja;
}
```

El arreglo claves permite guardar varios valores ordenados dentro del mismo nodo. El arreglo hijos permite representar los múltiples caminos posibles. La variable cantidadClaves indica cuántas posiciones del arreglo están efectivamente ocupadas. La variable hoja permite saber si el nodo tiene hijos o si se encuentra en el nivel final del árbol.

Esta representación muestra la diferencia central con el ABB y el AVL. En estructuras binarias, el nodo suele contener una clave y dos referencias. En el Árbol B, el nodo se parece más a una página de índice, porque concentra varias claves y varias referencias dentro de una misma unidad lógica.

Una implementación completa debe resolver búsqueda, inserción, división de nodos, promoción de claves y eliminación. En un desarrollo didáctico inicial, suele ser suficiente implementar búsqueda e inserción con división, y analizar la eliminación de manera conceptual por su mayor complejidad.

Síntesis conceptual

El Árbol B es una estructura de búsqueda balanceada y multicamino diseñada para trabajar eficientemente con grandes volúmenes de datos. Su principal aporte no consiste solo en mantener el orden, sino en reducir la altura del árbol mediante nodos que almacenan varias claves y permiten varios caminos de búsqueda.

La diferencia con ABB y AVL es fundamental. El ABB ofrece una organización binaria simple, pero puede desbalancearse. El AVL corrige el desbalance mediante rotaciones, pero sigue trabajando con nodos de una clave y dos hijos. El Árbol B cambia la lógica estructural: agrupa claves, aprovecha mejor los bloques de almacenamiento y reduce la cantidad de niveles necesarios para acceder a la información.

La inserción en Árbol B se basa en ubicar la hoja correspondiente, insertar la clave en orden y dividir el nodo si se supera su capacidad. La clave central se promociona al padre y el árbol conserva su balance. La eliminación agrega complejidad porque puede generar subocupación, lo que obliga a redistribuir claves mediante préstamo o a fusionar nodos.

El Árbol B+ extiende esta lógica con una organización especialmente útil para bases de datos: los nodos internos funcionan como índice y los datos se concentran en hojas enlazadas. Esto permite búsquedas puntuales eficientes y recorridos por rango, una

necesidad frecuente en sistemas empresariales, bancarios, logísticos, educativos y de auditoría.

La comprensión de estas estructuras permite tomar decisiones técnicas más precisas. No se trata de elegir el árbol “más avanzado”, sino de identificar qué problema se necesita resolver: ordenamiento simple, balance en memoria, reducción de accesos a disco o consultas eficientes por rango. En ese criterio de decisión se encuentra el verdadero valor del Árbol B dentro del diseño de estructuras de datos aplicadas a sistemas reales.